

Сокеты

```
int socket(int family, int type, int protocol)
// family: AF_UNIX, AF_INET, AF_INET6, AF_IPX, AF_UNSPEC
// type: SOCK_STREAM, SOCK_DGRAM, SOCK_RAW
// protocol: TCP,UDP
```

Листинг 0 — Сокеты

struct socket

```
struct socket {
    socket_state    state; /* SS_FREE, SS_UNCONNECTING, SS_CONNECTING,
SS_CONNECTED, SS_DISCONNECTING */

    short          type;

    unsigned long   flags;

    struct file     *file;
    struct sock     *sk;
    const struct proto_ops *ops; /* Might change with IPV6_ADDRFORM or
MPTCP. */

    struct socket_wq wq;
};

enum sock_type {
    SOCK_STREAM = 1, /* @SOCK_STREAM: stream (connection) socket */
    SOCK_DGRAM  = 2, /* @SOCK_DGRAM: datagram (conn.less) socket */
    SOCK_RAW    = 3, /* @SOCK_RAW: raw socket */
    SOCK_RDM    = 4,
    SOCK_SEQPACKET = 5,
    SOCK_DCCP    = 6,
    SOCK_PACKET = 10,
};
```

Листинг 1 — struct socket

Сокеты AF_INET, сетевой стек

```
struct sockaddr {
    sa_family_t sa_family; /* address family, AF_xxx */
    char        sa_data[14]; /* 14 bytes of protocol address */
};

struct sockaddr_in {
    __kernel_sa_family_t sin_family; /* Address family */
    __be16 sin_port; /* Port number */
    struct in_addr sin_addr; /* Internet address */

    /* Pad to size of `struct sockaddr'. */
    unsigned char __pad[__SOCK_SIZE__ - sizeof(short int) -
sizeof(unsigned short int) - sizeof(struct in_addr)];
};
```

```

struct in_addr {
    __be32  s_addr; //_u32
};

/* htons() --- host to network short */

uint16_t htons(uint16_t hostint16);
uint32_t htonl(uint32_t hostint32);

/* ntohs() --- network to host */

uint16_t ntohs(uint16_t netint16);
uint32_t ntohl(uint32_t netint32);

```

Листинг 2 — Сокеты AF_INET, сетевой стек

sys_socketcall

```

asmlinkage long sys_socketcall(int call, unsigned long *args)
{
    int err;
    if (copy_from_user(a, args, args[call]))
        return -EFAULT;

    a0 = a[0];
    a1 = a[1];

    switch (call)
    {
        case SYS_SOCKET:
            err = sys_socket(a0, a1, a[2]);
            break;
        case SYS_BIND:
            err = sys_bind(a0, (struct sockaddr *) a1, a[2]);
            break;
        case SYS_CONNECT:
            err = sys_connect(a0, (struct sockaddr *) a1, a[2]);
            break;
        ...
    }

    return err;
}

/* client: socket() -> [ bind() ] -> connect() -> write() -> close() */
/* server: socket() -> bind() -> listen() -> accept()->read()->close()*/

int  bind(int sockfd, struct sockaddr *addr, int addrlen)
int  connect(int sockfd, const struct sockaddr *serv_addr, int addrlen)
int  accept(int sockfd, struct sockaddr *addr, int *addrlen)
int  listen(int sockfd, int backlog)

```

Листинг 3 — sys_socketcall

sys_socket

```
asm linkage long sys_socket(int family, int type, int protocol)
{
    int retval;
    struct socket *sock;
    ...
    retval = socket_create(family, type, protocol, &sock);
    ...
    return retval;
}
```

Листинг 4 — sys_socket

Мультиплексор

```
struct epoll_event {
    uint32_t    events; /* Epoll events */
    epoll_data_t data; /* User data variable */
};

union epoll_data {
    void        *ptr;
    int          fd;
    uint32_t    u32;
    uint64_t    u64;
};
```

```
typedef union epoll_data    epoll_data_t;
```

```
int epoll_wait(int n, int epfd, struct epoll_event events[n], int n, int
timeout);
```

/* The epoll_wait() system call waits for events on the epoll(7) instance referred to by the file descriptor epfd. The buffer pointed to by events is used to return information from the ready list about file descriptors in the interest list that have some events available. Up to n are returned by epoll_wait(). The n argument must be greater than zero.

The timeout argument specifies the number of milliseconds that epoll_wait() will block. Time is measured against the CLOCK_MONOTONIC clock.

A call to epoll_wait() will block until either:

- a file descriptor delivers an event;
- the call is interrupted by a signal handler; or
- the timeout expires.

Note that the timeout interval will be rounded up to the system clock granularity, and kernel scheduling delays mean that the blocking interval may overrun by a small amount. Specifying a timeout of -1 causes epoll_wait() to block indefinitely, while specifying a timeout equal to zero causes epoll_wait() to return immediately, even if no events are available.

The struct epoll_event is described in epoll_event(3type).

The data field of each returned `epoll_event` structure contains the same data as was specified in the most recent call to `epoll_ctl(2)` (`EPOLL_CTL_ADD`, `EPOLL_CTL_MOD`) for the corresponding open file descriptor.

The events field is a bit mask that indicates the events that have occurred for the corresponding open file description. See `epoll_ctl(2)` for a list of the bits that may appear in this mask. */

```
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *_Nullable event);
```

/* This system call is used to add, modify, or remove entries in the interest list of the `epoll(7)` instance referred to by the file descriptor `epfd`. It requests that the operation `op` be performed for the target file descriptor, `fd`. Valid values for the `op` argument are:

`EPOLL_CTL_ADD`

Add an entry to the interest list of the `epoll` file descriptor, `epfd`. The entry includes the file descriptor, `fd`, a reference to the corresponding open file description (see `epoll(7)` and `open(2)`), and the settings specified in `event`.

`EPOLL_CTL_MOD`

Change the settings associated with `fd` in the interest list to the new settings specified in `event`.

`EPOLL_CTL_DEL`

Remove (deregister) the target file descriptor `fd` from the interest list. The `event` argument is ignored and can be `NULL` (but see `BUGS` below).

The `event` argument describes the object linked to the file descriptor `fd`. The `struct epoll_event` is described in `epoll_event(3type)`. The data member of the `epoll_event` structure specifies data that the kernel should save and then return (via `epoll_wait(2)`) when this file descriptor becomes ready.

The `events` member of the `epoll_event` structure is a bit mask composed by ORing together zero or more event types, returned by `epoll_wait(2)`, and input flags, which affect its behavior, but aren't returned. The available event types are:

`EPOLLIN`

The associated file is available for `read(2)` operations.

`EPOLLOUT`

The associated file is available for `write(2)` operations.

`EPOLLET`

Requests edge-triggered notification for the associated file descriptor. The default behavior for `epoll` is level-triggered. See `epoll(7)` for more detailed information about edge-triggered and level-triggered notification.

*/

struct super_block

```

struct super_block {
    struct list_head    s_list;    /* doubly linked lists in Linux */
    dev_t               s_dev;     /* search index; _not_ kdev_t */
    unsigned char       s_blocksize_bits;
    unsigned long       s_blocksize;
    loff_t              s_maxbytes; /* Max file size */
    struct file_system_type *s_type; /* pointer to FS type
(file_system_type) */
    const struct super_operations *s_op; /* pointer to superblock
operations (super_operations) */
    /* ... */
    unsigned long       s_flags;
    unsigned long       s_iflags; /* internal SB_I_* flags */
    unsigned long       s_magic; /* FS magic number */
    struct dentry       *s_root; /* mount point, root directory of mounted
FS */
    struct rw_semaphore s_umount; /* reader-writer semaphore for
mounting */
    /* ... */
    struct mount        *s_mounts; /* list of mounts of this FS (same FS
can be mounted multiple times) */
    struct block_device *s_bdev; /* associated block device (can go
away once we use an accessor for @s_bdev_file) */
    /* ... */
    struct list_head    s_inodes; /* list of all inodes of this FS */
    /* ... */
    struct list_head    s_inodes_wb; /* writeback inodes */
    /* ... */
}

struct super_operations {
    struct inode *(*alloc_inode)(struct super_block *sb); /* Creates
and initializes a new inode object for a specific mounted FS (allocates
address space) */
    void (*destroy_inode)(struct inode *inode); /* Removes inode from
the linked list of inodes */
    void (*free_inode)(struct inode *inode); /* Metadata (with inode
number) is not deleted; this inode can be reused */
    void (*dirty_inode)(struct inode *inode, int flags); /* Called to
move inode to another list if changes were made to it */
    int (*write_inode)(struct inode *inode, struct writeback_control
*wbc); /* Writes inode to disk (alloc alone is not enough) */
    int (*drop_inode)(struct inode *inode); /* Called by VFS interface
when the inode reference count reaches 0 */

    void (*put_super)(struct super_block *sb); /* Called by VFS when
unmounting FS; this superblock is removed from the mount list */

    int (*remount_fs)(struct super_block *sb, int *flags, char *data); /
* Called by VFS to remount the superblock with different parameters */

    void (*umount_begin)(struct super_block *sb); /* Called by VFS to

```

```

abort mounting; used by network FSes like NFS */
    ...
};

```

Листинг 6 — struct super_block

struct inode

```

struct inode {
    umode_t      i_mode; /* File type and permissions */
    unsigned short i_opflags;
    unsigned int  i_flags; /* Permissions and flags; flags act as
restrictions, indicating what can and cannot be done. */
    ...
    const struct inode_operations *i_op; /* Pointer to the inode operations
table (inode_operations). */
    struct super_block *i_sb; /* Pointer to the associated superblock: the
filesystem is mounted on a specific physical device, and any file belongs
to a specific FS. */
    struct address_space *i_mapping;
#ifdef CONFIG_SECURITY
    void *i_security;
#endif
    unsigned long i_ino; /* Inode number. */
    union {
        const unsigned int i_nlink ; /* Number of hard links. */
        unsigned int __i_nlink;
    }; /*
    * Filesystems may only read i_nlink directly. They shall use the
    * following functions for modification:
    *
    * (set|clear|inc|drop)_nlink
    * inode_(inc|dec)_link_count
    */

    dev_t      i_rdev;
    loff_t      i_size; /* File size; since files are associated with block
devices, the inode contains block size information (i_blkbits). */
    ...
    time64_t    i_atime_sec;
    time64_t    i_mtime_sec;
    time64_t    i_ctime_sec;
    u32         i_atime_nsec;
    u32         i_mtime_nsec;
    u32         i_ctime_nsec;
    ...
    u8         i_blkbits;
    ...
    struct rw_semaphore i_rwsem; /* Read-write semaphore. */
    ...
    struct hlist_node i_hash; /* Provides fast access to files (hash list).
*/
    ...
    struct list_head i_lru; /* Provides fast access to files (LRU list).
*/
    struct list_head i_sb_list; /* Provides fast access to files

```

```

(superblock list). */
struct list_head i_wb_list; /* backing dev writeback list */
union {
    struct hlist_head i_dentry;
    struct rcu_head i_rcu;
};
...
atomic_t i_readcount; /* Counter for the number of read accesses to
the file (via struct file). */
union {
    const struct file_operations *i_fop; /* former ->i_op-
>default_file_ops */
    void (*free_inode)(struct inode *);
};
...
union {
    struct pipe_inode_info *i_pipe;
    struct cdev *i_cdev;
    char *i_link;
    unsigned i_dir_seq;
};
}

```

Листинг 7 — struct inode

struct inode_operations

```

struct inode_operations {
    struct dentry * (*lookup) (struct inode *, struct dentry *, unsigned
int);
    const char * (*get_link) (struct dentry *, struct inode *, struct
delayed_call *);
    int (*permission) (struct mnt_idmap *, struct inode *, int);
    struct posix_acl * (*get_inode_acl)(struct inode *, int, bool);

    int (*readlink) (struct dentry *, char __user *, int);

    int (*create) (struct mnt_idmap *, struct inode *, struct dentry *,
umode_t, bool);
    int (*link) (struct dentry *, struct inode *, struct dentry *);
    int (*unlink) (struct inode *, struct dentry *);
    int (*symlink) (struct mnt_idmap *, struct inode *, struct dentry *,
const char *);
    struct dentry * (*mkdir) (struct mnt_idmap *, struct inode *,
struct dentry *, umode_t);
    int (*rmdir) (struct inode *, struct dentry *);
    int (*mknod) (struct mnt_idmap *, struct inode *, struct dentry *,
umode_t, dev_t);
    int (*rename) (struct mnt_idmap *, struct inode *, struct dentry *,
struct inode *, struct dentry *, unsigned int);
    int (*setattr) (struct mnt_idmap *, struct dentry *, struct iattr *);
    int (*getattr) (struct mnt_idmap *, const struct path *,
struct kstat *, u32, unsigned int);
    ssize_t (*listxattr) (struct dentry *, char *, size_t);
    int (*fiemap)(struct inode *, struct fiemap_extent_info *, u64 start,

```

```

        u64 len);
int (*update_time)(struct inode *inode, enum fs_update_time type,
    unsigned int flags);
void (*sync_lazytime)(struct inode *inode);
int (*atomic_open)(struct inode *, struct dentry *,
    struct file *, unsigned open_flag,
    umode_t create_mode);
int (*tmpfile)(struct mnt_idmap *, struct inode *,
    struct file *, umode_t);
struct posix_acl *(*get_acl)(struct mnt_idmap *, struct dentry *,
    int);
int (*set_acl)(struct mnt_idmap *, struct dentry *,
    struct posix_acl *, int);
int (*fileattr_set)(struct mnt_idmap *idmap,
    struct dentry *dentry, struct file_kattr *fa);
int (*fileattr_get)(struct dentry *dentry, struct file_kattr *fa);
struct offset_ctx *(*get_offset_ctx)(struct inode *inode);
}

```

Листинг 8 — struct inode_operations

struct dentry

```

struct dentry {
    /* RCU lookup touched fields */
    unsigned int d_flags; /* protected by d_lock */
    /* ... */
    struct hlist_bl_node d_hash; /* lookup hash list */
    struct dentry *d_parent; /* parent directory */
    struct qstr d_name;
    struct inode *d_inode; /* Where the name belongs to - NULL is
negative */
    /* ... */
    const struct dentry_operations *d_op; /* per-dentry operations listed
for each path element */
    struct super_block *d_sb; /* The root of the dentry tree - only for
mounted FS, access to its dirs/files; any dir/file belongs to a specific
FS */
    /* ... */
    struct list_head d_lru; /* LRU list */
    /* ... */
    struct list_head d_subdirs; /* our children */
    /* ... */
};

```

Листинг 9 — struct dentry

Функции для работы с кэшем SLAB

```

struct kmem_cache *kmem_cache_create(const char *name, size_t size,
size_t offset,
                                unsigned long flags, void
(*constructor)(void *));

void kmem_cache_destroy(struct kmem_cache *cache);
void *kmem_cache_alloc(struct kmem_cache *cache, gfp_t flags);

```

```
void kmem_cache_free(struct kmem_cache *cache, void *obj);
}
```

Листинг 10 — Функции для работы с кэшем SLAB

struct file_system_type

```
struct file_system_type {
    const char *name; /* File system name. */
    int fs_flags; /* File system type flags;
FS_REQUIRES_DEV is needed for FS that works with external storage (affects
mount functions). FS_USERSNS_MOUNT means not everything must be done in
kernel. */
    struct dentry *(*mount) (struct file_system_type *, int, const char *,
void *); /* Mount function. */
    void (*kill_sb) (struct super_block *); /* Unmount (kill superblock)
function. */
    struct module *owner; /* Owner module; needed for
reference counting (module cannot be unloaded while FS is mounted). */
    struct file_system_type *next; /* Linked list of
file_system_type. */
    struct hlist_head fs_supers; /* List of superblocks. */
};

extern int register_filesystem(struct file_system_type *);
extern int unregister_filesystem(struct file_system_type *);

extern struct dentry *mount_bdev(struct file_system_type *fs_type, int
flags,
    const char *dev_name, void *data, int (*fill_super)
(struct super_block *, void *, int));

extern struct dentry *mount_nodev(struct file_system_type *fs_type, int
flags, void *data, int (*fill_super) (struct super_block *, void *, int));

/* Mount function parameters:
* fs_type - filesystem type
* flags - mount flags
* dev_name - device name (only for mount_bdev; the external difference
between the two functions is this parameter)
* data - arbitrary mount data
* fill_super - pointer to the superblock filling function
*/
/* The functions differ significantly in code and scope of application:
* mount_bdev - for mounting a filesystem located on a block device
* mount_nodev - for mounting a filesystem not associated with any device
*/
struct vfsmount {
    struct dentry *mnt_root; /* root of the mounted tree */
    struct super_block *mnt_sb; /* pointer to superblock */
    int mnt_flags;
    struct mnt_idmap *mnt_idmap;
};
```

Листинг 11 — struct file_system_type

simple_fill_super

```
int simple_fill_super(struct super_block *s, unsigned long magic,
                     const struct tree_descr *files)
{
    struct inode *inode;
    struct dentry *dentry;
    int i;

    s->s_blocksize = PAGE_SIZE;
    s->s_blocksize_bits = PAGE_SHIFT;
    s->s_magic = magic;
    s->s_op = &simple_super_operations;
    s->s_time_gran = 1;

    inode = new_inode(s);
    if (!inode)
        return -ENOMEM;
    /*
     * because the root inode is 1, the files array must not contain an
     * entry at index 1
     */
    inode->i_ino = 1;
    inode->i_mode = S_IFDIR | 0755;
    simple_inode_init_ts(inode);
    inode->i_op = &simple_dir_inode_operations;
    inode->i_fop = &simple_dir_operations;
    set_nlink(inode, 2);
    s->s_root = d_make_root(inode);
    if (!s->s_root)
        return -ENOMEM;
    for (i = 0; !files->name || files->name[0]; i++, files++) {
        if (!files->name)
            continue;

        /* warn if it tries to conflict with the root inode */
        if (unlikely(i == 1))
            printk(KERN_WARNING "%s: %s passed in a files array"
                    "with an index of 1!\n", __func__,
                    s->s_type->name);

        dentry = d_alloc_name(s->s_root, files->name);
        if (!dentry)
            return -ENOMEM;
        inode = new_inode(s);
        if (!inode) {
            dput(dentry);
            return -ENOMEM;
        }
        inode->i_mode = S_IFREG | files->mode;
        simple_inode_init_ts(inode);
        inode->i_fop = files->ops;
        inode->i_ino = i;
        d_make_persistent(dentry, inode);
        dput(dentry);
    }
}
```

```

    return 0;
}

struct inode *new_inode(struct super_block *sb)
{
    struct inode *inode = new_inode_pseudo(sb); // -> alloc_inode(sb): from
slab
    if (inode)
        inode_sb_list_add(inode);
    return inode;
}

```

Листинг 12 — simple_fill_super

struct proc_dir_entry

```

struct proc_dir_entry {
    atomic_t in_use;                                /* atomic counter */
    ...
    struct list_head pde_openers;
    ...
    const struct inode_operations *proc_iops; /* inode operations for proc
file */
    union {
        const struct proc_ops *proc_ops;
        const struct file_operations *proc_dir_ops;
    }; /* function registration interface (proc_ops introduced to offload
overloaded file_operations) */
    ...
    const struct dentry_operations *proc_dops;
    union {
        const struct seq_operations *seq_ops;
        int (*single_show)(struct seq_file *, void *);
    }; /* seq interfaces: full (seq_ops) or simplified (single_show) */
    ...
    proc_write_t write; /* write callback (read removed, write remains
for user->kernel) */
    void *data; /* private data */
    unsigned int state_size;
    unsigned int low_ino; /* low-level inode (virtual FS, inode exists in
RAM only) */
    nlink_t nlink;
    ...
    loff_t size; /* file size (loff_t – typed for compile-time checking) */
    struct proc_dir_entry *parent; /* parent directory (maintains proc
hierarchy) */
    ...
    char *name;
    ...
    u8 namelen;
    char inline_name[];
};

```

Листинг 13 — struct proc_dir_entry

struct proc_ops

```
extern struct proc_dir_entry *proc_create_data(const char *, umode_t,
struct proc_dir_entry *, const struct proc_ops *, void *);

struct proc_dir_entry *proc_create(const char *name, umode_t mode, struct
proc_dir_entry *parent, const struct proc_ops *proc_ops);

extern struct proc_dir_entry *proc_mkdir(const char *, struct
proc_dir_entry *);

extern struct proc_dir_entry *proc_symlink(const char *, struct
proc_dir_entry *, const char *);

extern void remove_proc_entry(const char *, struct proc_dir_entry *);

struct proc_ops {
    unsigned int proc_flags;
    int (*proc_open)(struct inode *, struct file *);
    ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*proc_read_iter)(struct kiocb *, struct iov_iter *);
    ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t
*);
    /* mandatory unless nonseekable_open() or equivalent is used */
    loff_t (*proc_lseek)(struct file *, loff_t, int);
    int (*proc_release)(struct inode *, struct file *);
    __poll_t (*proc_poll)(struct file *, struct poll_table_struct *);
    long (*proc_ioctl)(struct file *, unsigned int, unsigned long);
#ifdef CONFIG_COMPAT
    long (*proc_compat_ioctl)(struct file *, unsigned int, unsigned long);
#endif
    int (*proc_mmap)(struct file *, struct vm_area_struct *);
    unsigned long (*proc_get_unmapped_area)(struct file *, unsigned long,
unsigned long, unsigned long, unsigned long);
} __randomize_layout;

static const struct proc_ops device_proc_ops = {
    .proc_open = device_open,
    .proc_read = device_read,
    .proc_write = device_write,
    .proc_release = device_release,
};

unsigned long __copy_to_user(void __user *to, const void *from, unsigned
long n);
unsigned long __copy_from_user(void *to, const void __user *from, unsigned
long n);
```

Листинг 14 — struct proc_ops

struct file

```
struct file {
    ...
    spinlock_t          f_lock;          /* Exclusive access to the file.
*/
```

```

    fmode_t          f_mode;          /* Access mode. */
    atomic_long_t    f_count;         /* File reference counter. */
    loff_t           f_pos;           /* Logical position in the file:
file is assumed to start at offset 0; each read/write operation changes
this field by the corresponding number of bytes. */

    ...
    struct path      f_path;          /* Link to dentry and mounted file
system. */
    struct inode      *f_inode;        /* Pointer to the inode (physical
file). */
    const struct file_operations *f_op; /* Operations defined on an open
file. */
    ...
};

```

Листинг 15 — struct file

struct file_operations

```

struct file_operations {
    struct module *owner;
    fop_flags_t fop_flags;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
    ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
    ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
    int (*iopoll)(struct kiocb *kiocb, struct io_comp_batch *,
        unsigned int flags);
    int (*iterate_shared) (struct file *, struct dir_context *);
    __poll_t (*poll) (struct file *, struct poll_table_struct *);
    long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
    long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *, fl_owner_t id);
    int (*release) (struct inode *, struct file *);
    int (*fsync) (struct file *, loff_t, loff_t, int datasync);
    int (*fasync) (int, struct file *, int);
    int (*lock) (struct file *, int, struct file_lock *);
    unsigned long (*get_unmapped_area)(struct file *, unsigned long,
unsigned long, unsigned long, unsigned long);
    int (*check_flags)(int);
    int (*flock) (struct file *, int, struct file_lock *);
    ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t
*, size_t, unsigned int);
    ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info
*, size_t, unsigned int);
    void (*splice_eof)(struct file *file);
    int (*setlease)(struct file *, int, struct file_lease **, void **);
    long (*fallocate)(struct file *file, int mode, loff_t offset,
        loff_t len);
    void (*show_fdinfo)(struct seq_file *m, struct file *f);
#ifdef CONFIG_MMU
    unsigned (*mmap_capabilities)(struct file *);
#endif

```

```

ssize_t (*copy_file_range)(struct file *, loff_t, struct file *,
                           loff_t, size_t, unsigned int);
loff_t (*remap_file_range)(struct file *file_in, loff_t pos_in,
                           struct file *file_out, loff_t pos_out,
                           loff_t len, unsigned int remap_flags);
int (*fadvise)(struct file *, loff_t, loff_t, int);
int (*uring_cmd)(struct io_uring_cmd *ioucmd, unsigned int issue_flags);
int (*uring_cmd_iopoll)(struct io_uring_cmd *, struct io_comp_batch *,
                        unsigned int poll_flags);
int (*mmap_prepare)(struct vm_area_desc *);
}

```

Листинг 16 — struct file_operations

open

```

int open(const char *pathname, int flags); /* pathname — Name of the file
to open. flags — Flags defining access mode and behavior. */
int open(const char *pathname, int flags, mode_t mode); /* mode — Access
permissions for the created file. The third parameter is required only
when open() is called to create a new file (with O_CREAT). If the file
already exists, mode is ignored. */

/* Returns a file descriptor — a non-negative integer, which is the lowest
 * unused descriptor in the process's open file table. Upon creation of a
new file, a new inode is initialized. When a file is opened, the struct
file fields (including f_pos) are initialized, with the file offset set to
the beginning of the file.
 */

/*
 * Access flags (one of them must be specified):
 *   O_RDONLY — Open file for reading only.
 *   O_WRONLY — Open file for writing only.
 *   O_RDWR  — Open file for reading and writing.
 *
 * Creation and behavior flags:
 *   O_CREAT   — Create the file if it does not exist. When this flag
is used, the mode parameter must be provided to set access permissions.
 *   O_APPEND  — Append to the existing file: before each write() call,
the file offset is moved to the end, so previously written data is
not overwritten.
 *   O_TRUNC   — If the file exists and is opened for writing, truncate
it to zero length.
 *   O_NONBLOCK — Set non-blocking mode. Useful for special files like
FIFOs and terminals; open() returns immediately instead of blocking.
Also affects read/write operations, allowing them to return
immediately with an error if no input is available or output cannot
be written.
 *   O_CLOEXEC  — Set the close-on-exec flag on the new file descriptor.
 *   O_PATH     — Obtain only a file descriptor (the file itself is not
actually opened for reading/writing).
 *   O_TMPFILE  — Create an unnamed temporary regular file.
 *
 * MODE (permissions for O_CREAT):
 * When creating a new file, the owner and group IDs are taken from the

```

```

* process's effective IDs. The following symbolic constants are
available:
*   S_IRWXU / S_IRWXG / S_IRWXO – Read, write, and execute permissions
    for owner / group / others.
*   S_IRUSR / S_IRGRP / S_IROTH – Read permission.
*   S_IWUSR / S_IWGRP / S_IWOTH – Write permission.
*   S_IXUSR / S_IXGRP / S_IXOTH – Execute permission.
*/

```

Листинг 17 — open

struct FILE

```

typedef struct _IO_FILE FILE;

FILE *fopen(const char *fname, const char *mode);

```

Листинг 18 — struct FILE

struct _IO_FILE

```

struct _IO_FILE {
    int _flags; /* High-order word is _IO_MAGIC; rest is flags. */
#define _IO_file_flags _flags

    /* The following pointers correspond to the C++ streambuf protocol. */
    /* Note: Tk uses the _IO_read_ptr and _IO_read_end fields directly. */
    char* _IO_read_ptr; /* Current read pointer */
    char* _IO_read_end; /* End of get area. */
    char* _IO_read_base; /* Start of putback+get area. */
    char* _IO_write_base; /* Start of put area. */
    char* _IO_write_ptr; /* Current put pointer. */
    char* _IO_write_end; /* End of put area. */
    char* _IO_buf_base; /* Start of reserve area. */
    char* _IO_buf_end; /* End of reserve area. */
    /* The following fields are used to support backing up and undo. */
    char *_IO_save_base; /* Pointer to start of non-current get area. */
    char *_IO_backup_base; /* Pointer to first valid character of backup
area */
    char *_IO_save_end; /* Pointer to end of non-current get area. */

    struct _IO_marker *_markers;

    struct _IO_FILE *_chain;

    int _fileno;
#if 0
    int _blksize;
#else
    int _flags2;
#endif
    _IO_off_t _old_offset; /* This used to be _offset but it's too small.
*/

#define __HAVE_COLUMN /* temporary */
    /* 1+column number of pbase(); 0 is unknown. */

```

```

unsigned short _cur_column;
signed char _vtable_offset;
char _shortbuf[1];

/* char* _save_gptr; char* _save_egptr; */

_IO_lock_t *_lock;
#ifdef _IO_USE_OLD_IO_FILE
};

```

Листинг 19 — struct IO_FILE

struct stat

```

struct stat {
    dev_t      st_dev;          /* [XSI] ID of device containing file */
    ino_t      st_ino;          /* [XSI] File serial number */
    mode_t     st_mode;         /* [XSI] Mode of file (see below) */
    nlink_t    st_nlink;        /* [XSI] Number of hard links */
    uid_t      st_uid;          /* [XSI] User ID of the file */
    gid_t      st_gid;          /* [XSI] Group ID of the file */
    dev_t      st_rdev;         /* [XSI] Device ID */
#ifdef _POSIX_C_SOURCE || defined(_DARWIN_C_SOURCE)
    struct timespec st_atimespec; /* time of last access */
    struct timespec st_mtimespec; /* time of last data modification */
    struct timespec st_ctimespec; /* time of last status change */
#else
    time_t      st_atime;        /* [XSI] Time of last access */
    long        st_atimensec;    /* nsec of last access */
    time_t      st_mtime;        /* [XSI] Last data modification time */
    long        st_mtimensec;    /* last data modification nsec */
    time_t      st_ctime;        /* [XSI] Time of last status change */
    long        st_ctimensec;    /* nsec of last status change */
#endif
    off_t       st_size;         /* [XSI] file size, in bytes */
    blkcnt_t    st_blocks;       /* [XSI] blocks allocated for file */
    blksize_t   st_blksize;      /* [XSI] optimal blocksize for I/O */
    __uint32_t  st_flags;        /* user defined flags for file */
    __uint32_t  st_gen;          /* file generation number */
    __int32_t   st_lspare;       /* RESERVED: DO NOT USE! */
    __int64_t   st_qspare[2];    /* RESERVED: DO NOT USE! */
};

```

Листинг 20 — struct stat

Обработчики аппаратных прерываний

```

/**
 * enum irqreturn - irqreturn type values
 * @IRQ_NONE:   interrupt was not from this device or was not handled
 * @IRQ_HANDLED: interrupt was handled by this device
 * @IRQ_WAKE_THREAD: handler requests to wake the handler thread
 */
enum irqreturn {
    IRQ_NONE      = (0 << 0),
    IRQ_HANDLED   = (1 << 0),

```

```

IRQ_WAKE_THREAD    = (1 << 1),
};

typedef enum irqreturn irqreturn_t;
}

```

Листинг 21 — Обработчики аппаратных прерываний

Флаги IRQ

```

/*
 * These flags used only by the kernel as part of the
 * irq handling routines.
 *
 * IRQF_SHARED - allow sharing the irq among several devices
 * IRQF_PROBE_SHARED - set by callers when they expect sharing mismatches
to occur
 * IRQF_TIMER - Flag to mark this interrupt as timer interrupt
 * IRQF_PERCPU - Interrupt is per cpu
 * IRQF_NOBALANCING - Flag to exclude this interrupt from irq balancing
 * IRQF_IRQPOLL - Interrupt is used for polling (only the interrupt that
is
 *                  registered first in a shared interrupt is considered for
 *                  performance reasons)
 * IRQF_ONESHOT - Interrupt is not reenabled after the hardirq handler
finished.
 *                  Used by threaded interrupts which need to keep the
 *                  irq line disabled until the threaded handler has been
run.
 * IRQF_NO_SUSPEND - Do not disable this IRQ during suspend. Does not
guarantee
 *                  that this interrupt will wake the system from a
suspended
 *                  state. See Documentation/power/suspend-and-
interrupts.rst
 * IRQF_FORCE_RESUME - Force enable it on resume even if IRQF_NO_SUSPEND
is set
 * IRQF_NO_THREAD - Interrupt cannot be threaded
 * IRQF_EARLY_RESUME - Resume IRQ early during syscore instead of at
device
 *                  resume time.
 * IRQF_COND_SUSPEND - If the IRQ is shared with a NO_SUSPEND user,
execute this
 *                  interrupt handler after suspending interrupts. For
system
 *                  wakeup devices users need to implement wakeup detection
in
 *                  their interrupt handlers.
 * IRQF_NO_AUTOEN - Don't enable IRQ or NMI automatically when users
request it.
 *                  Users will enable it explicitly by enable_irq() or
enable_nmi()
 *                  later.
 * IRQF_NO_DEBUG - Exclude from runaway detection for IPI and similar
handlers,
 *                  depends on IRQF_PERCPU.

```

```

* IRQF_COND_ONESHOT - Agree to do IRQF_ONESHOT if already set for a
shared
*
*          interrupt.
*/
#define IRQF_SHARED      0x00000080
#define IRQF_PROBE_SHARED 0x00000100
#define __IRQF_TIMER      0x00000200
#define IRQF_PERCPU      0x00000400
#define IRQF_NOBALANCING  0x00000800
#define IRQF_IRQPOLL      0x00001000
#define IRQF_ONESHOT      0x00002000
#define IRQF_NO_SUSPEND   0x00004000
#define IRQF_FORCE_RESUME 0x00008000
#define IRQF_NO_THREAD     0x00010000
#define IRQF_EARLY_RESUME 0x00020000
#define IRQF_COND_SUSPEND 0x00040000
#define IRQF_NO_AUTOEN     0x00080000
#define IRQF_NO_DEBUG      0x00100000
#define IRQF_COND_ONESHOT 0x00200000

```

Листинг 22 — Флаги IRQ

struct irqaction

```

/**
 * struct irqaction - per interrupt action descriptor
 * @handler: interrupt handler function
 * @name: name of the device
 * @dev_id: cookie to identify the device
 * @percpu_dev_id: cookie to identify the device
 * @affinity: CPUs this irqaction is allowed to run on
 * @next: pointer to the next irqaction for shared interrupts
 * @irq: interrupt number
 * @flags: flags (see IRQF_* above)
 * @thread_fn: interrupt handler function for threaded interrupts
 * @thread: thread pointer for threaded interrupts
 * @secondary: pointer to secondary irqaction (force threading)
 * @thread_flags: flags related to @thread
 * @thread_mask: bitmask for keeping track of @thread activity
 * @dir: pointer to the proc/irq/NN/name entry
 */
struct irqaction {
    irq_handler_t handler;
    union {
        void *dev_id;
        void __percpu *percpu_dev_id;
    };
    const struct cpumask *affinity;
    struct irqaction *next;
    irq_handler_t thread_fn;
    struct task_struct *thread;
    struct irqaction *secondary;
    unsigned int irq;
    unsigned int flags;
    unsigned long thread_flags;
    unsigned long thread_mask;

```

```

    const char    *name;
    struct proc_dir_entry *dir;
}

/**
 * request_irq - Add a handler for an interrupt line
 * @irq: The interrupt line to allocate
 * @handler: Function to be called when the IRQ occurs.
 *           Primary handler for threaded interrupts
 *           If NULL, the default primary handler is installed
 * @flags: Handling flags
 * @name: Name of the device generating this interrupt
 * @dev: A cookie passed to the handler function
 *
 * This call allocates an interrupt and establishes a handler; see
 * the documentation for request_threaded_irq() for details.
 */

static inline int __must_check
request_irq(unsigned int irq, irq_handler_t handler, unsigned long flags,
            const char *name, void *dev)
{
    return request_threaded_irq(irq, handler, NULL, flags |
IRQF_COND_ONESHOT, name, dev);
}

extern const void *free_irq(unsigned int, void *); /* free irq line from
handler */

```

Листинг 23 — struct irqaction

softirq

```

/* PLEASE, avoid to allocate new softirqs, if you need not _really_ high
frequency threaded job scheduling. For almost all the purposes
tasklets are more than enough. F.e. all serial device BHs et
al. should be converted to tasklets, not to softirqs.
*/

enum
{
    HI_SOFTIRQ=0,
    TIMER_SOFTIRQ,
    NET_TX_SOFTIRQ, /* network */
    NET_RX_SOFTIRQ, /* network */
    BLOCK_SOFTIRQ,
    IRQ_POLL_SOFTIRQ,
    TASKLET_SOFTIRQ,
    SCHED_SOFTIRQ,
    HRTIMER_SOFTIRQ,
    RCU_SOFTIRQ, /* Preferable RCU should always be the last softirq */

    NR_SOFTIRQS
};

```

```

struct softirq_action {
    void (*action)(struct softirq_action *); /* executable function */
};

static struct softirq_action softirq_vec[NR_SOFTIRQS];

const char * const softirq_to_name[NR_SOFTIRQS] = {
    "HI", "TIMER", "NET_TX", "NET_RX", "BLOCK", "IRQ_POLL", "TASKLET",
    "SCHED", "HRTIMER", "RCU"
};

void raise_softirq(unsigned int nr); /* Marks softirq as pending on
current CPU (called from HW interrupt handler) */

void open_softirq(int nr, void (*action)(void)); /* Registers softirq
handler (called during initialization) */

/*
 * open_softirq - registers handler (nr = index in softirq_vec, action =
handler function)
 * raise_softirq - marks registered softirq as pending so it can be
executed
 */

```

Листинг 24 — softirq

tasklet

```

/* Tasklets --- multithreaded analogue of BHs.

   This API is deprecated. Please consider using threaded IRQs instead:
   https://lore.kernel.org/lkml/20200716081538.2sivhkj4hcyrusem@
linutronix.de

   Main feature differing them of generic softirqs: tasklet
   is running only on one CPU simultaneously.

   Main feature differing them of BHs: different tasklets
   may be run simultaneously on different CPUs.

   Properties:
   * If tasklet_schedule() is called, then tasklet is guaranteed
     to be executed on some cpu at least once after this.
   * If the tasklet is already scheduled, but its execution is still not
     started, it will be executed only once.
   * If this tasklet is already running on another CPU (or schedule is
called
     from tasklet itself), it is rescheduled for later.
   * Tasklet is strictly serialized wrt itself, but not
     wrt another tasklets. If client needs some intertask synchronization,
     he makes it with spinlocks.
 */

struct tasklet_struct
{
    struct tasklet_struct *next;

```

```

    unsigned long state;
    atomic_t count;
    bool use_callback;
    union {
        void (*func)(unsigned long data);
        void (*callback)(struct tasklet_struct *t);
    };
    unsigned long data;
};

enum
{
    TASKLET_STATE_SCHED, /* Tasklet is scheduled for execution */
    TASKLET_STATE_RUN /* Tasklet is running (SMP only) */
};

#define DECLARE_TASKLET(name, _callback) \
struct tasklet_struct name = { \
    .count = ATOMIC_INIT(0), \
    .callback = _callback, \
    .use_callback = true, \
}

#define DECLARE_TASKLET_DISABLED(name, _callback) \
struct tasklet_struct name = { \
    .count = ATOMIC_INIT(1), \
    .callback = _callback, \
    .use_callback = true, \
}

#define from_tasklet(var, callback_tasklet, tasklet_fieldname) \
    container_of(callback_tasklet, typeof(*var), tasklet_fieldname)

void tasklet_init(struct tasklet_struct *t,
    void (*func)(unsigned long), unsigned long data)
{
    t->next = NULL;
    t->state = 0;
    atomic_set(&t->count, 0);
    t->func = func;
    t->use_callback = false;
    t->data = data;
}

static void __tasklet_schedule_common(struct tasklet_struct *t,
    struct tasklet_head __percpu *headp,
    unsigned int softirq_nr)
{
    struct tasklet_head *head;
    unsigned long flags;

    local_irq_save(flags);
    head = this_cpu_ptr(headp);

```

```

t->next = NULL;
*head->tail = t;
head->tail = &(t->next);
raise_softirq_irqoff(softirq_nr);
local_irq_restore(flags);
}

void __tasklet_schedule(struct tasklet_struct *t)
{
    __tasklet_schedule_common(t, &tasklet_vec,
                              TASKLET_SOFTIRQ);
}
EXPORT_SYMBOL(__tasklet_schedule);

void __tasklet_hi_schedule(struct tasklet_struct *t)
{
    __tasklet_schedule_common(t, &tasklet_hi_vec,
                              HI_SOFTIRQ);
}
EXPORT_SYMBOL(__tasklet_hi_schedule);

```

Листинг 25 — tasklet

struct worker

```

struct worker_pool;

/*
 * The poor guys doing the actual heavy lifting. All on-duty workers are
 * either serving the manager role, on idle list or on busy hash. For
 * details on the locking annotation (L, I, X...), refer to workqueue.c.
 *
 * Only to be used in workqueue and async.
 */
struct worker {
    /* on idle list while idle, on busy hash table while busy */
    union {
        struct list_head entry; /* L: while idle */
        struct hlist_node hentry; /* L: while busy */
    };

    struct work_struct *current_work; /* K: work being processed and its
    */
    work_func_t current_func; /* K: function */
    struct pool_workqueue *current_pwq; /* K: pwq */
    u64 current_at; /* K: runtime at start or last wakeup */
    unsigned long current_start; /* K: start time of current work item */
    unsigned int current_color; /* K: color */

    int sleeping; /* S: is worker sleeping? */

    /* used by the scheduler to determine a worker's last known identity */
    work_func_t last_func; /* K: last work's fn */

    struct list_head scheduled; /* L: scheduled works */

```

```

struct task_struct *task; /* I: worker task */
struct worker_pool *pool; /* A: the associated pool */
/* L: for rescuers */
struct list_head node; /* A: anchored at pool->workers */
/* A: runs through worker->node */

unsigned long last_active; /* K: last active timestamp */
unsigned int flags; /* L: flags */
int id; /* I: worker id */

/*
 * Opaque string set with work_set_desc(). Printed out with task
 * dump for debugging - WARN, BUG, panic or sysrq.
 */
char desc[WORKER_DESC_LEN];

/* used only by rescuers to point to the target workqueue */
struct workqueue_struct *rescue_wq; /* I: the workqueue to rescue */
};

```

Листинг 26 — struct worker

struct worker_pool

```

struct worker_pool {
    raw_spinlock_t lock; /* the pool lock */
    int cpu; /* I: the associated cpu */
    int node; /* I: the associated node ID */
    int id; /* I: pool ID */
    unsigned int flags; /* L: flags */

    unsigned long last_progress_ts; /* L: last forward progress timestamp */
    bool cpu_stall; /* WD: stalled cpu bound pool */

    /*
     * The counter is incremented in a process context on the associated CPU
     * w/ preemption disabled, and decremented or reset in the same context
     * but w/ pool->lock held. The readers grab pool->lock and are
     * guaranteed to see if the counter reached zero.
     */
    int nr_running;

    struct list_head worklist; /* L: list of pending works */

    int nr_workers; /* L: total number of workers */
    int nr_idle; /* L: currently idle workers */

    struct list_head idle_list; /* L: list of idle workers */
    struct timer_list idle_timer; /* L: worker idle timeout */
    struct work_struct idle_cull_work; /* L: worker idle cleanup */

    struct timer_list mayday_timer; /* L: SOS timer for workers */

    /* a workers is either on busy_hash or idle_list, or the manager */
    DECLARE_HASHTABLE(busy_hash, BUSY_WORKER_HASH_ORDER);
};

```

```

        /* L: hash of busy workers */

    struct worker    *manager; /* L: purely informational */
    struct list_head  workers; /* A: attached workers */

    struct ida        worker_ida; /* worker IDs for task name */

    struct workqueue_attrs *attrs; /* I: worker attributes */
    struct hlist_node hash_node; /* PL: unbound_pool_hash node */
    int               refcnt; /* PL: refcnt for unbound pools */
#ifdef CONFIG_PREEMPT_RT
    spinlock_t        cb_lock; /* BH worker cancel lock */
#endif
    /*
     * Destruction of pool is RCU protected to allow dereferences
     * from get_work_pool().
     */
    struct rcu_head    rcu;
};

```

Листинг 27 — struct worker_pool

Инициализация очереди работ

```

#define DECLARE_WORK(n, f) \
    struct work_struct n = __WORK_INITIALIZER(n, f)

#define __INIT_WORK(_work, _func, _onstack) \
    do { \
        static __maybe_unused struct lock_class_key __key; \
        __INIT_WORK_KEY(_work, _func, _onstack, &__key); \
    } while (0)

#define INIT_WORK(_work, _func) \
    __INIT_WORK((_work), (_func), 0)

int queue_work(struct workqueue_struct *wq, struct work_struct *work);
int queue_work_on(int cpu, struct workqueue_struct *wq, struct work_struct *work);
int queue_delayed_work(struct workqueue_struct *wq, struct work_struct *work, unsigned long delay);

```

Листинг 28 — Инициализация очереди работ

Выделение и освобождение номеров устройств

```

extern int alloc_chrdev_region(dev_t *, unsigned, unsigned, const char *);
extern int register_chrdev_region(dev_t, unsigned, const char *);
extern int __register_chrdev(unsigned int major, unsigned int baseminor,
    unsigned int count, const char *name,
    const struct file_operations *fops);
extern void __unregister_chrdev(unsigned int major, unsigned int baseminor, unsigned int count, const char *name);
extern void unregister_chrdev_region(dev_t, unsigned);
extern void chrdev_show(struct seq_file *, off_t);

```

```

static inline int register_chrdev(unsigned int major, const char *name,
                                const struct file_operations *fops)
{
    return __register_chrdev(major, 0, 256, name, fops);
}

static inline void unregister_chrdev(unsigned int major, const char *name)
{
    __unregister_chrdev(major, 0, 256, name);
}

MAJOR(dev_t dev); /* get major number */
MINOR(dev_t dev); /* get minor number */

/* Having major and minor numbers, they can be combined into dev_t: */
MKDEV(int major, int minor); /* combine into dev_t */

```

Листинг 29 — Выделение и освобождение номеров устройств

Загрузка модуля

```

#include <linux/module.h>
#include <linux/init.h>

static int __init md_init(void)
{
    printk("Module loaded!\n");
    return 0;
}

static int __init md_init(void)
{
    struct task_struct *task = &init_task;
    do {
        printk(KERN_INFO "Task: pid: %d, comm: %s, ppid: %d, state: %ld,
flags: %x, prio: %d, policy: %d, root: %s, pwd: %s\n",
               task->pid, task->comm, task->parent->pid,
               task->state, task->flags, task->prio, task->policy,
               task->fs->root.dentry->d_iname, task->fs->pwd.dentry-
>d_name.name);
    } while ((task = next_task(task)) != &init_task);
    printk(KERN_INFO "Current: pid: %d, comm: %s, ppid: %d, state: %ld,
flags: %x, prio: %d, policy: %d, root: %s, pwd: %s\n",
           current->pid, current->comm, current->parent->pid,
           current->state, current->flags, current->prio, current->policy,
           current->fs->root.dentry->d_iname, current->fs->pwd.dentry-
>d_name.name);
    return 0;
}

static void __exit md_exit(void)
{
    printk("Module unloaded\n");
}

```

```

    module_init(md_init);
    module_exit(md_exit);
}

```

Листинг 30 — Загрузка модуля

printk flags

```

#define KERN_EMERG   KERN_SOH "0" /* system is unusable */
#define KERN_ALERT   KERN_SOH "1" /* action must be taken immediately */
#define KERN_CRIT    KERN_SOH "2" /* critical conditions */
#define KERN_ERR      KERN_SOH "3" /* error conditions */
#define KERN_WARNING  KERN_SOH "4" /* warning conditions */
#define KERN_NOTICE   KERN_SOH "5" /* normal but significant condition */
#define KERN_INFO     KERN_SOH "6" /* informational */
#define KERN_DEBUG    KERN_SOH "7" /* debug-level messages */
}

```

Листинг 31 — printk flags

task_struct

```

struct task_struct {
#ifdef CONFIG_THREAD_INFO_IN_TASK
    /*
     * For reasons of header soup (see current_thread_info()), this
     * must be the first element of task_struct.
     */
    struct thread_info    thread_info;
#endif
    unsigned int          __state;

    /* saved state for "spinlock sleepers" */
    unsigned int          saved_state;

    /*
     * This begins the randomizable portion of task_struct. Only
     * scheduling-critical items should be added above here.
     */
    randomized_struct_fields_start

    void                  *stack;
    refcount_t            usage;
    /* Per task flags (PF_*), defined further below: */
    unsigned int          flags;
    unsigned int          ptrace;

#ifdef CONFIG_MEM_ALLOC_PROFILING
    struct alloc_tag      *alloc_tag;
#endif

    int                   on_cpu;
    struct __call_single_node wake_entry;
    unsigned int          wakee_flips;
    unsigned long          wakee_flip_decay_ts;

```

```

struct task_struct    *last_wakee;

/*
 * recent_used_cpu is initially set as the last CPU used by a task
 * that wakes affine another task. Waker/wakee relationships can
 * push tasks around a CPU where each wakeup moves to the next one.
 * Tracking a recently used CPU allows a quick search for a recently
 * used CPU that may be idle.
 */
int        recent_used_cpu;
int        wake_cpu;
int        on_rq;

int        prio;
int        static_prio;
int        normal_prio;
unsigned int    rt_priority;

struct sched_entity    se;
struct sched_rt_entity    rt;
struct sched_dl_entity    dl;
struct sched_dl_entity    *dl_server;
#ifdef CONFIG_SCHED_CLASS_EXT
struct sched_ext_entity    scx;
#endif
const struct sched_class    *sched_class;

#ifdef CONFIG_SCHED_CORE
struct rb_node    core_node;
unsigned long    core_cookie;
unsigned int    core_occupation;
#endif

#ifdef CONFIG_CGROUP_SCHED
struct task_group    *sched_task_group;
#endif
#ifdef CONFIG_CFS_BANDWIDTH
struct callback_head    sched_throttle_work;
struct list_head    throttle_node;
bool    throttled;
#endif
#endif

#ifdef CONFIG_UCLAMP_TASK
/*
 * Clamp values requested for a scheduling entity.
 * Must be updated with task_rq_lock() held.
 */
struct uclamp_se    uclamp_req[UCLAMP_CNT];
/*
 * Effective clamp values used for a scheduling entity.
 * Must be updated with task_rq_lock() held.
 */
struct uclamp_se    uclamp[UCLAMP_CNT];
#endif

```

```

    struct sched_statistics      stats;

#ifdef CONFIG_PREEMPT_NOTIFIERS
    /* List of struct preempt_notifier: */
    struct hlist_head    preempt_notifiers;
#endif

#ifdef CONFIG_BLK_DEV_IO_TRACE
    unsigned int        btrace_seq;
#endif

    unsigned int        policy;
    unsigned long        max_allowed_capacity;
    int                nr_cpus_allowed;
    const cpumask_t      *cpus_ptr;
    cpumask_t          *user_cpus_ptr;
    cpumask_t          cpus_mask;
    void                *migration_pending;
    unsigned short       migration_disabled;
    unsigned short       migration_flags;

#ifdef CONFIG_PREEMPT_RCU
    int                rcu_read_lock_nesting;
    union rcu_special    rcu_read_unlock_special;
    struct list_head     rcu_node_entry;
    struct rcu_node      *rcu_blocked_node;
#endif /* #ifdef CONFIG_PREEMPT_RCU */

#ifdef CONFIG_TASKS_RCU
    unsigned long        rcu_tasks_nvcsw;
    u8                  rcu_tasks_holdout;
    u8                  rcu_tasks_idx;
    int                rcu_tasks_idle_cpu;
    struct list_head     rcu_tasks_holdout_list;
    int                rcu_tasks_exit_cpu;
    struct list_head     rcu_tasks_exit_list;
#endif /* #ifdef CONFIG_TASKS_RCU */

#ifdef CONFIG_TASKS_TRACE_RCU
    int                trc_reader_nesting;
    struct srcu_ctr      __percpu *trc_reader_scp;
#endif /* #ifdef CONFIG_TASKS_TRACE_RCU */

    struct sched_info    sched_info;

    struct list_head     tasks;
    struct plist_node     pushable_tasks;
    struct rb_node        pushable_dl_tasks;

    struct mm_struct      *mm;
    struct mm_struct      *active_mm;

    int                exit_state;
    int                exit_code;
    int                exit_signal;
    /* The signal sent when the parent dies: */

```

```

int      pdeath_signal;
/* JOBCTL_*, siglock protected: */
unsigned long  jobctl;

/* Used for emulating ABI behavior of previous Linux versions: */
unsigned int   personality;

/* Scheduler bits, serialized by scheduler locks: */
unsigned       sched_reset_on_fork:1;
unsigned       sched_contributes_to_load:1;
unsigned       sched_migrated:1;
unsigned       sched_task_hot:1;

/* Force alignment to the next boundary: */
unsigned       :0;

/* Unserialized, strictly 'current' */

/*
 * This field must not be in the scheduler word above due to wakelist
 * queueing no longer being serialized by p->on_cpu. However:
 *
 * p->XXX = X;      ttwu()
 * schedule()      if (p->on_rq && ..) // false
 *   smp_mb__after_spinlock();   if (smp_load_acquire(&p->on_cpu) && //
true
 *   deactivate_task()          ttwu_queue_wakelist())
 *   p->on_rq = 0;      p->sched_remote_wakeup = Y;
 *
 * guarantees all stores of 'current' are visible before
 * ->sched_remote_wakeup gets used, so it can be in this word.
 */
unsigned       sched_remote_wakeup:1;
#ifdef CONFIG_RT_MUTEXES
unsigned       sched_rt_mutex:1;
#endif

/* Save user-dumpable when mm goes away */
unsigned       user_dumpable:1;

/* Bit to tell TOMOYO we're in execve(): */
unsigned       in_execve:1;
unsigned       in_iowait:1;
#ifdef TIF_RESTORE_SIGMASK
unsigned       restore_sigmask:1;
#endif
#ifdef CONFIG_MEMCG_V1
unsigned       in_user_fault:1;
#endif
#ifdef CONFIG_LRU_GEN
/* whether the LRU algorithm may apply to this access */
unsigned       in_lru_fault:1;
#endif
#ifdef CONFIG_COMPAT_BRK
unsigned       brk_randomized:1;
#endif

```

```

#ifdef CONFIG_CGROUPS
    /* disallow userland-initiated cgroup migration */
    unsigned    no_cgroup_migration:1;
    /* task is frozen/stopped (used by the cgroup freezer) */
    unsigned    frozen:1;
#endif
#ifdef CONFIG_BLK_CGROUP
    unsigned    use_memdelay:1;
#endif
#ifdef CONFIG_PSI
    /* Stalled due to lack of memory */
    unsigned    in_memstall:1;
#endif
#ifdef CONFIG_PAGE_OWNER
    /* Used by page_owner=on to detect recursion in page tracking. */
    unsigned    in_page_owner:1;
#endif
#ifdef CONFIG_EVENTFD
    /* Recursion prevention for eventfd_signal() */
    unsigned    in_eventfd:1;
#endif
#ifdef CONFIG_ARCH_HAS_CPU_PASID
    unsigned    pasid_activated:1;
#endif
#ifdef CONFIG_X86_BUS_LOCK_DETECT
    unsigned    reported_split_lock:1;
#endif
#ifdef CONFIG_TASK_DELAY_ACCT
    /* delay due to memory thrashing */
    unsigned    in_thrashing:1;
#endif
    unsigned    in_nf_duplicate:1;
#ifdef CONFIG_PREEMPT_RT
    struct netdev_xmit    net_xmit;
#endif
    unsigned long    atomic_flags; /* Flags requiring atomic access. */

    struct restart_block    restart_block;

    pid_t    pid;
    pid_t    tgid;

#ifdef CONFIG_STACKPROTECTOR
    /* Canary value for the -fstack-protector GCC feature: */
    unsigned long    stack_canary;
#endif
    /*
     * Pointers to the (original) parent process, youngest child, younger
     sibling,
     * older sibling, respectively. (p->father can be replaced with
     * p->real_parent->pid)
     */

    /* Real parent process: */
    struct task_struct    __rcu    *real_parent;

```

```

/* Recipient of SIGCHLD, wait4() reports: */
struct task_struct __rcu *parent;

/*
 * Children/sibling form the list of natural children:
 */
struct list_head children;
struct list_head sibling;
struct task_struct *group_leader;

/*
 * 'ptraced' is the list of tasks this task is using ptrace() on.
 *
 * This includes both natural children and PTRACE_ATTACH targets.
 * 'ptrace_entry' is this task's link on the p->parent->ptraced list.
 */
struct list_head ptraced;
struct list_head ptrace_entry;

/* PID/PID hash table linkage. */
struct pid *thread_pid;
struct hlist_node pid_links[PIDTYPE_MAX];
struct list_head thread_node;

struct completion *vfork_done;

/* CLONE_CHILD_SETTID: */
int __user *set_child_tid;

/* CLONE_CHILD_CLEARTID: */
int __user *clear_child_tid;

/* PF_KTHREAD | PF_IO_WORKER */
void *worker_private;

u64 utime;
u64 stime;
#ifdef CONFIG_ARCH_HAS_SCALED_CPUTIME
u64 utimescaled;
u64 stimescaled;
#endif
u64 gtime;
struct prev_cputime prev_cputime;
#ifdef CONFIG_VIRT_CPU_ACCOUNTING_GEN
struct vtime vtime;
#endif

#ifdef CONFIG_NO_HZ_FULL
atomic_t tick_dep_mask;
#endif
/* Context switch counts: */
unsigned long nvcs;
unsigned long nivcs;

/* Monotonic time in nsecs: */
u64 start_time;

```

```

/* Boot based time in nsecs: */
u64      start_boottime;

/* MM fault and swap info: this can arguably be seen as either mm-
specific or thread-specific: */
unsigned long    min_flt;
unsigned long    maj_flt;

/* Empty if CONFIG_POSIX_CPUTIMERS=n */
struct posix_cputimers    posix_cputimers;

#ifdef CONFIG_POSIX_CPU_TIMERS_TASK_WORK
    struct posix_cputimers_work    posix_cputimers_work;
#endif

/* Process credentials: */

/* Tracer's credentials at attach: */
const struct cred __rcu    *ptracer_cred;

/* Objective and real subjective task credentials (COW): */
const struct cred __rcu    *real_cred;

/* Effective (overridable) subjective task credentials (COW): */
const struct cred __rcu    *cred;

#ifdef CONFIG_KEYS
/* Cached requested key. */
struct key    *cached_requested_key;
#endif

/*
 * executable name, excluding path.
 *
 * - normally initialized begin_new_exec()
 * - set it with set_task_comm()
 * - strcpy_pad() to ensure it is always NUL-terminated and
 *   zero-padded
 * - task_lock() to ensure the operation is atomic and the name is
 *   fully updated.
 */
char    comm[TASK_COMM_LEN];

struct nameidata    *nameidata;

#ifdef CONFIG_SYSVIPC
    struct sysv_sem    sysvsem;
    struct sysv_shm    sysvshm;
#endif
#ifdef CONFIG_DETECT_HUNG_TASK
    unsigned long    last_switch_count;
    unsigned long    last_switch_time;
#endif
/* Filesystem information: */
struct fs_struct    *fs;

```

```

/* Open file information: */
struct files_struct    *files;

#ifdef CONFIG_IO_URING
    struct io_uring_task    *io_uring;
    struct io_restriction    *io_uring_restrict;
#endif

/* Namespaces: */
struct nsproxy        *nsproxy;

/* Signal handlers: */
struct signal_struct    *signal;
struct sighand_struct    __rcu    *sighand;
sigset_t                blocked;
sigset_t                real_blocked;
/* Restored if set_restore_sigmask() was used: */
sigset_t                saved_sigmask;
struct sigpending        pending;
unsigned long            sas_ss_sp;
size_t                  sas_ss_size;
unsigned int             sas_ss_flags;

struct callback_head    *task_works;

#ifdef CONFIG_AUDIT
#ifdef CONFIG_AUDITSYSCALL
    struct audit_context    *audit_context;
#endif
    kuid_t                loginuid;
    unsigned int            sessionid;
#endif
    struct seccomp            seccomp;
    struct syscall_user_dispatch    syscall_dispatch;

/* Thread group tracking: */
u64                parent_exec_id;
u64                self_exec_id;

/* Protection against (de-)allocation: mm, files, fs, tty, keyrings,
mems_allowed, mempolicy: */
spinlock_t            alloc_lock;

/* Protection of the PI data structures: */
raw_spinlock_t            pi_lock;

struct wake_q_node        wake_q;

#ifdef CONFIG_RT_MUTEXES
/* PI waiters blocked on a rt_mutex held by this task: */
struct rb_root_cached    pi_waiters;
/* Updated under owner's pi_lock and rq lock */
struct task_struct        *pi_top_task;
/* Deadlock detection and priority inheritance handling: */
struct rt_mutex_waiter    *pi_blocked_on;

```

```

#endif

    struct mutex      *blocked_on; /* lock we're blocked on */

#ifdef CONFIG_DETECT_HUNG_TASK_BLOCKER
/*
 * Encoded lock address causing task block (lower 2 bits = type from
 * <linux/hung_task.h>). Accessed via hung_task_*() helpers.
 */
    unsigned long      blocker;
#endif

#ifdef CONFIG_DEBUG_ATOMIC_SLEEP
    int      non_block_count;
#endif

#ifdef CONFIG_TRACE_IRQFLAGS
    struct irqtrace_events      irqtrace;
    unsigned int      hardirq_threaded;
    u64      hardirq_chain_key;
    int      softirqs_enabled;
    int      softirq_context;
    int      irq_config;
#endif

#ifdef CONFIG_PREEMPT_RT
    int      softirq_disable_cnt;
#endif

#ifdef CONFIG_LOCKDEP
# define MAX_LOCK_DEPTH      48UL
    u64      curr_chain_key;
    int      lockdep_depth;
    unsigned int      lockdep_recursion;
    struct held_lock      held_locks[MAX_LOCK_DEPTH];
#endif

#if defined(CONFIG_UBSAN) && !defined(CONFIG_UBSAN_TRAP)
    unsigned int      in_ubsan;
#endif

/* Journalling filesystem info: */
void      *journal_info;

/* Stacked block device info: */
struct bio_list      *bio_list;

/* Stack plugging: */
struct blk_plug      *plug;

/* VM state: */
struct reclaim_state      *reclaim_state;

struct io_context      *io_context;

#ifdef CONFIG_COMPACTION
    struct capture_control      *capture_control;

```

```

#endif
/* Ptrace state: */
unsigned long    ptrace_message;
kernel_siginfo_t *last_siginfo;

    struct task_io_accounting ioac;
#ifdef CONFIG_PSI
/* Pressure stall state */
unsigned int     psi_flags;
#endif
#ifdef CONFIG_TASK_XACCT
/* Accumulated RSS usage: */
u64             acct_rss_mem1;
/* Accumulated virtual memory usage: */
u64             acct_vm_mem1;
/* stime + utime since last update: */
u64             acct_timexpd;
#endif
#ifdef CONFIG_CPUSETS
/* Protected by ->alloc_lock: */
nodemask_t      mems_allowed;
/* Sequence number to catch updates: */
seqcount_spinlock_t mems_allowed_seq;
int             cpuset_mem_spread_rotor;
#endif
#ifdef CONFIG_CGROUPS
/* Control Group info protected by css_set_lock: */
struct css_set __rcu *cgroups;
/* cg_list protected by css_set_lock and tsk->alloc_lock: */
struct list_head cg_list;
#ifdef CONFIG_PREEMPT_RT
    struct llist_node cg_dead_lnode;
#endif /* CONFIG_PREEMPT_RT */
#endif /* CONFIG_CGROUPS */
#ifdef CONFIG_X86_CPU_RESCTRL
    u32             closid;
    u32             rmid;
#endif
#ifdef CONFIG_FUTEX
    struct robust_list_head __user *robust_list;
#endif
#ifdef CONFIG_COMPAT
    struct compat_robust_list_head __user *compat_robust_list;
#endif
    struct list_head pi_state_list;
    struct futex_pi_state *pi_state_cache;
    struct mutex      futex_exit_mutex;
    unsigned int      futex_state;
#endif
#ifdef CONFIG_PERF_EVENTS
    u8             perf_recursion[PERF_NR_CONTEXTS];
    struct perf_event_context *perf_event_ctxp;
    struct mutex      perf_event_mutex;
    struct list_head   perf_event_list;
    struct perf_ctx_data __rcu *perf_ctx_data;
#endif
#ifdef CONFIG_DEBUG_PREEMPT

```

```

    unsigned long    preempt_disable_ip;
#endif
#ifdef CONFIG_NUMA
    /* Protected by alloc_lock: */
    struct mempolicy *mempolicy;
    short    il_prev;
    u8    il_weight;
    short    pref_node_fork;
#endif
#ifdef CONFIG_NUMA_BALANCING
    int    numa_scan_seq;
    unsigned int    numa_scan_period;
    unsigned int    numa_scan_period_max;
    int    numa_preferred_nid;
    unsigned long    numa_migrate_retry;
    /* Migration stamp: */
    u64    node_stamp;
    u64    last_task_numa_placement;
    u64    last_sum_exec_runtime;
    struct callback_head    numa_work;

    /*
     * This pointer is only modified for current in syscall and
     * pagefault context (and for tasks being destroyed), so it can be read
     * from any of the following contexts:
     * - RCU read-side critical section
     * - current->numa_group from everywhere
     * - task's runqueue locked, task not running
     */
    struct numa_group __rcu    *numa_group;

    /*
     * numa_faults is an array split into four regions:
     * faults_memory, faults_cpu, faults_memory_buffer, faults_cpu_buffer
     * in this precise order.
     *
     * faults_memory: Exponential decaying average of faults on a per-node
     * basis. Scheduling placement decisions are made based on these
     * counts. The values remain static for the duration of a PTE scan.
     * faults_cpu: Track the nodes the process was running on when a NUMA
     * hinting fault was incurred.
     * faults_memory_buffer and faults_cpu_buffer: Record faults per node
     * during the current scan window. When the scan completes, the counts
     * in faults_memory and faults_cpu decay and these values are copied.
     */
    unsigned long    *numa_faults;
    unsigned long    total_numa_faults;

    /*
     * numa_faults_locality tracks if faults recorded during the last
     * scan window were remote/local or failed to migrate. The task scan
     * period is adapted based on the locality of the faults with different
     * weights depending on whether they were shared or private faults
     */
    unsigned long    numa_faults_locality[3];

```

```

    unsigned long    numa_pages_migrated;
#endif /* CONFIG_NUMA_BALANCING */

    struct rseq_data    rseq;
    struct sched_mm_cid    mm_cid;

    struct tlbflush_unmap_batch tlb_ubic;

    /* Cache last used pipe for splice(): */
    struct pipe_inode_info    *splice_pipe;

    struct page_frag    task_frag;

#ifdef CONFIG_ARCH_HAS_LAZY_MMU_MODE
    struct lazy_mmu_state    lazy_mmu_state;
#endif

#ifdef CONFIG_TASK_DELAY_ACCT
    struct task_delay_info    *delays;
#endif

#ifdef CONFIG_FAULT_INJECTION
    int    make_it_fail;
    unsigned int    fail_nth;
#endif
    /*
     * When (nr_dirtied >= nr_dirtied_pause), it's time to call
     * balance_dirty_pages() for a dirty throttling pause:
     */
    int    nr_dirtied;
    int    nr_dirtied_pause;
    /* Start of a write-and-pause period: */
    unsigned long    dirty_paused_when;

#ifdef CONFIG_LATENCYTOP
    int    latency_record_count;
    struct latency_record    latency_record[LT_SAVECOUNT];
#endif
    /*
     * Time slack values; these are used to round up poll() and
     * select() etc timeout values. These are in nanoseconds.
     */
    u64    timer_slack_ns;
    u64    default_timer_slack_ns;

#ifdef CONFIG_KASAN_GENERIC || defined(CONFIG_KASAN_SW_TAGS)
    unsigned int    kasan_depth;
#endif

#ifdef CONFIG_KCSAN
    struct kcsan_ctx    kcsan_ctx;
#endif
#ifdef CONFIG_TRACE_IRQFLAGS
    struct irqtrace_events    kcsan_save_irqtrace;
#endif
#ifdef CONFIG_KCSAN_WEAK_MEMORY
    int    kcsan_stack_depth;

```

```

#endif
#endif

#ifdef CONFIG_KMSAN
    struct kmsan_ctx    kmsan_ctx;
#endif

#if IS_ENABLED(CONFIG_KUNIT)
    struct kunit        *kunit_test;
#endif

#ifdef CONFIG_FUNCTION_GRAPH_TRACER
    /* Index of current stored address in ret_stack: */
    int        curr_ret_stack;
    int        curr_ret_depth;

    /* Stack of return addresses for return function tracing: */
    unsigned long    *ret_stack;

    /* Timestamp for last schedule: */
    unsigned long long    ftrace_timestamp;
    unsigned long long    ftrace_sleeptime;

    /*
     * Number of functions that haven't been traced
     * because of depth overrun:
     */
    atomic_t        trace_overrun;

    /* Pause tracing: */
    atomic_t        tracing_graph_pause;
#endif

#ifdef CONFIG_TRACING
    /* Bitmask and counter of trace recursion: */
    unsigned long    trace_recursion;
#endif /* CONFIG_TRACING */

#ifdef CONFIG_KCOV
    /* See kernel/kcov.c for more details. */

    /* Coverage collection mode enabled for this task (0 if disabled): */
    unsigned int        kcov_mode;

    /* Size of the kcov_area: */
    unsigned int        kcov_size;

    /* Buffer for coverage collection: */
    void        *kcov_area;

    /* KCOV descriptor wired with this task or NULL: */
    struct kcov        *kcov;

    /* KCOV common handle for remote coverage collection: */
    u64        kcov_handle;

```

```

/* KCOV sequence number: */
int      kcov_sequence;

/* Collect coverage from softirq context: */
unsigned int      kcov_softirq;
#endif

#ifdef CONFIG_MEMCG_V1
    struct mem_cgroup      *memcg_in_oom;
#endif

#ifdef CONFIG_MEMCG
    /* Number of pages to reclaim on returning to userland: */
    unsigned int      memcg_nr_pages_over_high;

    /* Used by memcontrol for targeted memcg charge: */
    struct mem_cgroup      *active_memcg;

    /* Cache for current->cgroups->memcg->objcg lookups: */
    struct obj_cgroup      *objcg;
#endif

#ifdef CONFIG_BLK_CGROUP
    struct gendisk      *throttle_disk;
#endif

#ifdef CONFIG_UPROBES
    struct uprobe_task      *utask;
#endif
#if defined(CONFIG_BCACHE) || defined(CONFIG_BCACHE_MODULE)
    unsigned int      sequential_io;
    unsigned int      sequential_io_avg;
#endif
    struct kmap_ctrl      kmap_ctrl;
#ifdef CONFIG_DEBUG_ATOMIC_SLEEP
    unsigned long      task_state_change;
# ifdef CONFIG_PREEMPT_RT
    unsigned long      saved_state_change;
# endif
#endif
    struct rcu_head      rcu;
    refcount_t      rcu_users;
    int      pagefault_disabled;
#ifdef CONFIG_MMU
    struct task_struct      *oom_reaper_list;
    struct timer_list      oom_reaper_timer;
#endif
#ifdef CONFIG_VMAP_STACK
    struct vm_struct      *stack_vm_area;
#endif
#ifdef CONFIG_THREAD_INFO_IN_TASK
    /* A live task holds one reference: */
    refcount_t      stack_refcount;
#endif
#ifdef CONFIG_LIVEPATCH
    int      patch_state;

```

```

#endif
#ifdef CONFIG_SECURITY
    /* Used by LSM modules for access restriction: */
    void          *security;
#endif
#ifdef CONFIG_BPF_SYSCALL
    /* Used by BPF task local storage */
    struct bpf_local_storage __rcu *bpf_storage;
    /* Used for BPF run context */
    struct bpf_run_ctx      *bpf_ctx;
#endif
    /* Used by BPF for per-TASK xdp storage */
    struct bpf_net_context   *bpf_net_context;

#ifdef CONFIG_KSTACK_ERASE
    unsigned long    lowest_stack;
#endif
#ifdef CONFIG_KSTACK_ERASE_METRICS
    unsigned long    prev_lowest_stack;
#endif

#ifdef CONFIG_RANDOMIZE_KSTACK_OFFSET
    u32              kstack_offset;
#endif

#ifdef CONFIG_X86_MCE
    void __user      *mce_vaddr;
    __u64            mce_kflags;
    u64              mce_addr;
    __u64            mce_ripv : 1,
                mce_whole_page : 1,
                __mce_reserved : 62;
    struct callback_head mce_kill_me;
    int              mce_count;
#endif

#ifdef CONFIG_KRETPROBES
    struct llist_head      kretprobe_instances;
#endif
#ifdef CONFIG_RETHOOK
    struct llist_head      rethooks;
#endif

#ifdef CONFIG_ARCH_HAS_PARANOID_L1D_FLUSH
    /*
     * If L1D flush is supported on mm context switch
     * then we use this callback head to queue kill work
     * to kill tasks that are not running on SMT disabled
     * cores
     */
    struct callback_head    l1d_flush_kill;
#endif

#ifdef CONFIG_RV
    /*
     * Per-task RV monitor, fixed in CONFIG_RV_PER_TASK_MONITORS.

```

```

    * If memory becomes a concern, we can think about a dynamic method.
    */
    union rv_task_monitor    rv[CONFIG_RV_PER_TASK_MONITORS];
#endif

#ifdef CONFIG_USER_EVENTS
    struct user_event_mm    *user_event_mm;
#endif

#ifdef CONFIG_UNWIND_USER
    struct unwind_task_info    unwind_info;
#endif

    /* CPU-specific state of this task: */
    struct thread_struct    thread;

    /*
     * New fields for task_struct should be added above here, so that
     * they are included in the randomized portion of task_struct.
     */
    randomized_struct_fields_end
} __attribute__((aligned (64)));

```

Листинг 32 — task_struct